# CloudGuard Azure
# Cloud Security Posture Assessment Project Report

Azure resource assessment, custom storage checks and historical evidence

Project Version: 1.0

Report Date: 1 November 2026

Section: AIML-TIGERS [2023-2027]


**Submitted By:**

*Anohita Sen - 23WU0102016*
*Aashutosh Gautam - 23WU0102005*
*Diganta Das - 23WU0102051*
*Deepak Kumar - 23WU0102050*
*D D S S Mahith - 23WU0102045*

**Guided By:**

*Dr. Karthick N.*
*Assistant Professor, SOT*
*Woxsen University, Hyderabad*


This report explains the design, implementation and observed results of CloudGuard Azure, including the evidence needed to assess its current release readiness.

## Index (Table of Contents)


1 Abstract4
1.1 Project overview4
1.2 Principal findings4
2 Introduction5
2.1 Background and motivation5
2.2 Problem statement5
2.3 Project scope and users5
2.4 Azure laboratory environment5
3 Technology Review6
3.1 Microsoft Defender for Cloud6
3.2 Azure Resource Graph and authentication6
3.3 Python and supporting libraries6
3.4 SQLite and portable evidence formats7
3.5 Testing tools and local dashboard7
4 Review Summary7
4.1 Lessons from the technology review7
4.2 Comparison of review approaches7
4.3 Design implications8
5 Limitations or Research Gap8
5.1 Gap addressed by the project8
5.2 Coverage and evaluation limits8
5.3 Source and database consistency8
5.4 Incomplete collection and deployment limits8
6 Objectives9
6.1 Primary objective9
6.2 Functional objectives9
6.3 Quality and operational objectives9
7 Methodology10
7.1 Incremental development approach10
7.2 System architecture10
7.3 Configuration and data collection10
7.4 Finding normalisation and identity11
7.5 Custom storage rule evaluation11
7.6 Collection reliability and error handling12
7.7 Snapshot persistence and data relationships12
7.8 Comparison and export procedure12
7.9 Verification method13
8 Novelty13
8.1 Source-aware assessment records13
8.2 Explicit treatment of uncertainty13
8.3 Traceable observations over time14
9 Results/Outcome14
9.1 Live assessment results14
9.2 Custom rule outcome and interpretation14
9.3 Historical testing outcomes15
9.4 Repository outcomes and remaining acceptance criteria15
10 Conclusion15
10.1 Project contribution15
10.2 Completion assessment15
11 Recommendation and Future Scope16
11.1 Immediate release recommendations16
11.2 Controlled remediation validation16
11.3 Broader coverage and usability16
11.4 Future evaluation and deployment16
12 Acknowledgment17
12.1 Technical articles and community resources17
12.2 Project documentation and open-source tools17
13 References17
13.1 Project materials17
13.2 Official documentation and technical blog17
14 Word Limit**Error! Bookmark not defined.**
14.1 Report word count**Error! Bookmark not defined.**

## List of Figures and Tables

Table 1  Technology stack and repository requirements6
Table 2  Roles of the assessment approaches7
Table 3  Functional objectives and assessment criteria9
Figure 1  CloudGuard architecture and evidence flow10
Table 4  Custom storage rule definitions11
Figure 2  Evidence-based custom rule decision11
Figure 3  Version 2 snapshot and finding relationships12
Figure 4  Comparison decision for an existing open finding13
Table 5  Saved live assessment outcome14
Table 6  Historical verification milestones15

# 1 Abstract

## 1.1 Project overview

CloudGuard Azure is a Python-based Cloud Security Posture Management (CSPM) project that brings Azure resource inventory, Microsoft Defender for Cloud assessments and custom storage checks into a single evidence workflow. The project addresses a practical problem: a security recommendation is difficult to act on when its source, affected scope or supporting configuration is unclear. CloudGuard retains these details, saves assessment snapshots locally and compares findings across runs. Its purpose is to support informed review of cloud configuration rather than to make automatic changes to Azure resources.

The backend uses Azure Resource Graph for collection, Azure Identity for authentication, explicit Python rules for assessment, and SQLite for persistence. Four custom rules examine HTTPS-only traffic, anonymous blob access, minimum TLS version and the storage network ACL default action. Missing or unrecognised configuration values remain UNKNOWN. Microsoft recommendations retain their own source identity and assessed resource scope, which prevents subscription-level recommendations from being misrepresented as storage-account findings. [1]

## 1.2 Principal findings

The documented live baseline contains one discovered resource, 22 Defender assessments and four custom findings. Ten Defender assessments and one custom check are unhealthy. The custom failure concerns a network ACL default action of Allow, while the project rule requires Deny. Later evidence in the working folder, recorded at 18:22:42 UTC on 30 October 2026, shows the same totals. These observations describe configuration posture; they do not establish a security incident. [1, 3]

The project demonstrates collection, rule evaluation, evidence export and historical storage. Its release readiness remains limited by a source/database version mismatch and incomplete propagation of collection reliability into comparison. A local HTML, CSS and JavaScript dashboard is also present, served by Python, but its presence does not remove these backend limitations. The main contribution is a traceable assessment workflow with explicit treatment of uncertainty and source identity. [1, 2]

# 2 Introduction

## 2.1 Background and motivation

Cloud resources can be created quickly, but their security settings still require deliberate review. A storage account may enforce secure transport while allowing a broad network perimeter. Subscription-level recommendations may appear alongside resource-specific observations. Looking at a single healthy setting therefore gives only a partial view of the environment.

CSPM helps identify configuration conditions that deserve attention. Microsoft Defender for Cloud supplies native recommendations, while a project may also need a smaller set of transparent checks that can be inspected and tested directly. CloudGuard combines these two sources without obscuring who produced each finding. The resulting record connects a condition to its resource, evidence and observation time. Microsoft's CSPM overview provides the wider technical context for this approach. [4]

## 2.2 Problem statement

The project asks how a small Azure assessment tool can collect security information, apply understandable rules and retain enough context to compare results safely. Three difficulties drive the design. Native recommendations may target different scopes. Configuration properties may be absent or ambiguous. A later scan may return less data because collection failed, rather than because a problem was fixed.

A useful report must keep these situations separate. CloudGuard therefore records the source and assessed resource of each finding, preserves unknown values and models collection outcomes independently from security outcomes. This is especially important when reporting improvement: the disappearance of a record cannot, by itself, prove remediation. [1]

## 2.3 Project scope and users

The principal scope is the backend work described in Phases A to D and Features G and H. It covers the Azure lab baseline, collection, custom checks, finding models, evidence files, SQLite snapshots and comparison exports. The intended users are students, project evaluators and operators who need to understand the evidence behind a configuration assessment.

The working repository also contains a local dashboard extension. It offers another way to inspect stored findings and start a scan through the existing backend. It is discussed as a repository component, separate from the backend completion scope of the primary documentation. Automated remediation, enterprise multi-tenant operation and exhaustive Azure security coverage are outside the demonstrated scope. [1, 2]

## 2.4 Azure laboratory environment

The documented environment uses an Azure for Students subscription with Foundational CSPM enabled. The resource group is rg-cloudguard-v1, and the test storage account is storagecloudguard in Central India. The live inventory contains one storage account. This small environment makes individual rule decisions easy to explain, although it cannot establish large-scale performance or broad security coverage. [1, 3]

The first baseline recorded 14 Defender assessments on 24 October 2026. The later baseline recorded 22. These are observations from different times, not contradictory totals for a single scan. Subscription identifiers are unnecessary for understanding the report and are not reproduced here.

# 3 Technology Review

## 3.1 Microsoft Defender for Cloud

Defender for Cloud is the native source of security assessment records in this project. CloudGuard preserves the original assessment ID, status, severity and assessed resource ID instead of converting every recommendation into a custom rule. This preserves the meaning of Microsoft's assessment while allowing it to appear in a common finding model. [1]

Foundational CSPM and the separately priced Defender CSPM plan are distinct offerings. The laboratory baseline establishes Foundational CSPM, not activation of every paid workload protection plan. A recommendation concerning an additional plan must be reviewed in that context; an unhealthy recommendation is not evidence that a storage account has been compromised. [4]

## 3.2 Azure Resource Graph and authentication

Azure Resource Graph provides the query interface for resource inventory, security assessments and storage properties. Its query language is based on Kusto Query Language. Querying returned resource properties through a common interface suits the project's small collector modules and reduces the need for a different service client for every property. Resource Graph is eventually consistent, so a configuration change may not be reflected immediately. [5]

The azure-identity package supplies DefaultAzureCredential, and azure-mgmt-resourcegraph supplies ResourceGraphClient and query request models. Azure Identity can use supported developer or hosted credentials; the documented local workflow uses Azure CLI sign-in. Authentication identifies the caller, while Azure permissions determine which resources the caller can read. A successful sign-in alone does not establish complete visibility. [1, 5, 7]

## 3.3 Python and supporting libraries

Python is used for orchestration, collectors, rule evaluation, model definitions and persistence. Dataclasses and enums express the data contract, while argparse handles command-line options. The json and csv modules produce portable outputs, hashlib generates stable finding keys, and datetime records UTC observations. python-dotenv loads local configuration. The historical verification baseline is Python 3.12.0; the README specifies Python 3.11 or newer. A release should select and test one explicit runtime baseline. [1, 2]

**Table 1 Technology stack and repository requirements**

| Technology | Version or basis | Project responsibility |
|---|---|---|
| Python | 3.12.0 historical baseline | Backend and local dashboard server |
| azure-identity | >= 1.17.0 | Credential acquisition |
| azure-mgmt-resourcegraph | >= 8.0.0 | Resource Graph queries |
| python-dotenv | >= 1.0.0 | Local environment configuration |
| pytest and pytest-cov | >= 8.0.0 and >= 5.0.0 | Automated tests and coverage |
| SQLite and sqlite3 | Python standard-library interface | Local snapshots and observation history |
| HTML, CSS and JavaScript | Repository frontend files | Browser presentation and interactions |

The package values in Table 1 are minimum constraints from requirements.txt, not confirmed installed versions. The backend does not depend on pandas, Streamlit, Flask, a hosted SQL server, Docker or Kubernetes. Introducing those technologies into the description would misrepresent the implementation. [1, 2]

## 3.4 SQLite and portable evidence formats

SQLite stores data in a local file and does not require a separate database server. This suits a project that needs persistent assessment history without provisioning another cloud service. Python's sqlite3 interface provides the database operations. Within CloudGuard, one snapshot represents one run, while a finding can have observations in several runs. [1, 8]

JSON retains nested evidence and raw assessment fields. CSV provides a flattened comparison for spreadsheet review; the exporter uses UTF-8 with a byte-order mark for compatibility. These formats serve different purposes: JSON supports detailed processing, CSV supports manual review, and SQLite supports historical retrieval. None of them substitutes for interpreting the underlying finding. [1]

## 3.5 Testing tools and local dashboard

pytest is the project's test runner, and pytest-cov supplies coverage reporting. The repository separates tests for models, collectors, storage rules, persistence, comparison, exports and orchestration. This division makes it possible to test a rule with controlled input without making a live Azure request. pytest's fixture and temporary-directory facilities support this style of isolated verification. [1, 9]

The dashboard uses plain HTML, CSS and JavaScript with a Python standard-library HTTP server. Its source defaults to the loopback address 127.0.0.1 and port 8765. It reads snapshot data and comparison results, and includes a scan endpoint that invokes the backend. This is a local project interface; the repository inspection does not establish production hosting, remote authentication or multi-user deployment. [2]

# 4 Review Summary

## 4.1 Lessons from the technology review

The reviewed technologies support a small and understandable architecture. Azure services provide the native inventory and recommendation data. Python provides explicit transformations and rule logic. SQLite keeps history local, and JSON/CSV exports allow results to be examined outside the application. The Microsoft storage guidance offers context for transport, access and network controls, but the implemented coverage is limited to the four project rules. [1, 6]

The Microsoft Defender for Cloud Data Flow article is useful background for understanding how recommendations can be consumed beyond the portal. CloudGuard applies this general idea in a local assessment workflow, while adding its own rule evidence and snapshot comparison. This is an engineering adaptation, not evidence that the project replaces the wider capabilities of Defender for Cloud. [10]

## 4.2 Comparison of review approaches

**Table 2 Roles of the assessment approaches**

| Approach | Useful contribution | Boundary in this project |
|---|---|---|
| Azure portal review | Direct examination of resource settings | Manual observations need consistent recording |
| Native Defender assessments | Microsoft-originated recommendations and severity | Scope and applicability must be retained |
| CloudGuard custom rules | Visible conditions and evidence for four storage checks | Limited rule coverage |
| CloudGuard snapshots | Stored observations and cross-run comparison | Depends on schema compatibility and collection completeness |
| Local dashboard | Accessible view of stored project results | Depends on the backend and selected database |

The approaches complement one another. Portal review helps explain the environment; native assessments supply broader recommendations; custom rules make a small control set reproducible. Snapshot comparison adds value only when the evidence is sufficiently complete to support the claimed transition.

### 4.3 Design implications

The review supports retaining source labels, explicit status values and raw evidence throughout the workflow. It also supports keeping the rule engine separate from collectors so that rule decisions can be tested with known inputs. The outstanding design obligation is to carry collection completeness through every stage. Without that connection, a careful comparison function can still receive misleading defaults. [1, 2]

## 5 Limitations or Research Gap

### 5.1 Gap addressed by the project

The project addresses a practical integration gap between retrieving cloud recommendations and explaining how a finding changes over time. A list of recommendations does not automatically preserve source identity, distinguish missing evidence or prove that a later healthy state is trustworthy. CloudGuard brings those concerns into the data model and comparison design.

This contribution should be described at project scale. The reviewed evidence does not establish a new detection algorithm, a previously unknown security vulnerability or superiority over commercial CSPM products. Its value lies in making a limited assessment process transparent and testable.

### 5.2 Coverage and evaluation limits

The live laboratory contains one resource and four custom storage checks. It does not demonstrate coverage of virtual machines, Key Vault, databases, identity permissions or all storage security controls. Missing soft-delete fields, for example, are retained as unknown evidence and are not evaluated by the current rules. No measured precision, recall, false-positive rate, throughput or large-subscription benchmark is available. [1, 3]

The historical automated results relate to particular development stages. They cannot establish that the current source tree passes every test, nor can package coverage demonstrate that every Azure service failure has been exercised. Controlled tests for throttling, pagination, incomplete data and permissions remain necessary.

### 5.3 Source and database consistency

The current snapshot_store.py declares schema version 2. The original data/cloudguard.db reports version 3 and contains three snapshots; the source rejects databases newer than its supported version. A separate live-output/cloudguard.db reports version 2 and contains ten snapshots. These two stores must not be described as one database with a single compatibility state. [1, 2, 3]

The newer README describes reliability behaviour that is not fully reflected in the inspected collector and pipeline code. Resource inventory and Defender collection still return lists, while storage uses CollectionResult. The pipeline does not pass actual completeness flags to compare_snapshots. The report therefore treats the source as evidence of implementation and the README as a description that still requires reconciliation. [2]

### 5.4 Incomplete collection and deployment limits

The storage collector's exception branch reports the number of records already received but does not pass the accumulated record list into CollectionResult. If a later page fails after earlier records were collected, that construction conflicts with the model's count validation. Preserving partial evidence and testing this path are necessary before claiming reliable partial-run handling. [1, 2]

The local dashboard also depends on database compatibility and truthful collection metadata. A functioning presentation layer cannot certify that a scan was complete. Remote deployment would require a separate design for authentication, authorisation, protected configuration and process isolation; these capabilities are not established by the local implementation.

# 6 Objectives

## 6.1 Primary objective

The primary objective is to build an understandable Azure posture assessment workflow that connects each security finding to its source, scope and configuration evidence, then preserves that information for later review. A successful implementation should allow an evaluator to explain why a finding exists and what evidence would be needed to confirm a change.

## 6.2 Functional objectives

**Table 3 Functional objectives and assessment criteria**

| Objective | Evidence required |
|---|---|
| Discover resources | Inventory records retain resource ID, type, group and location |
| Collect native assessments | Defender IDs, statuses, severities and assessed scopes are preserved |
| Evaluate storage rules | Each CG-SA rule returns a supported status and observed evidence |
| Retain assessment history | A unique run can be saved and retrieved without losing its payload |
| Compare observations | Stable identities connect findings across runs and preserve uncertainty |
| Export reviewable results | JSON and CSV identify the runs, findings, classifications and evidence |

## 6.3 Quality and operational objectives

The quality objectives are to handle missing values explicitly, distinguish collection failure from security failure, retain valid partial data, and avoid unsupported resolution claims. Operational objectives include an offline demonstration mode, manageable local dependencies and read-oriented Azure collection. Release acceptance requires a compatible database, reproducible outputs, a clean test run and proof that incomplete evidence cannot resolve an open finding. [1]

# 7 Methodology

## 7.1 Incremental development approach

Development progressed from an Azure baseline to a Python collection pipeline, then to rule correctness and persistence. Phase A established the laboratory resources. Phase B implemented the proof of concept and evidence generation. Phase C strengthened custom checks so that absent or ambiguous input produces UNKNOWN. Phase D added snapshots, comparison and exports. Features G and H introduced reliability contracts and a richer finding model. [1]

This progression allowed the project to check one concern at a time. Establishing inventory first made it possible to verify collection. Defining rule outcomes next made persistence meaningful. Historical comparison then exposed the need to distinguish an unchanged finding from a missing observation. Integration of the reliability contract remains an unfinished part of that progression.

## 7.2 System architecture

The CLI is the orchestration entry point. It loads configuration, chooses live or mock mode, gathers data, evaluates storage checks, writes evidence and saves a snapshot. When an earlier snapshot exists, the pipeline creates comparison exports. The dashboard is a separate local consumer of the backend's saved state and can initiate a new backend scan. Figure 1 shows these relationships; it does not imply that every reliability safeguard is already integrated. [1, 2]

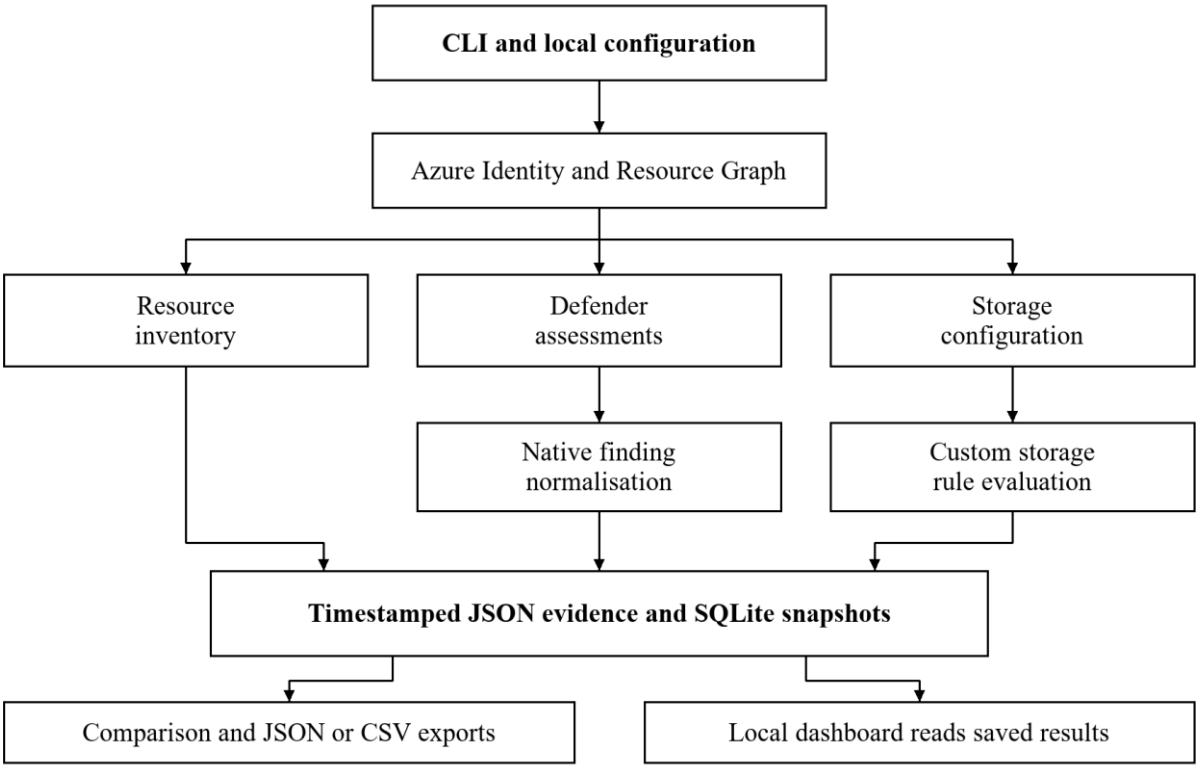



**Figure 1 CloudGuard architecture and evidence flow**

Native Defender assessments and custom findings meet at the finding model while retaining distinct source values. This is necessary because their rule ownership and assessed scope differ. The storage configuration is also retained independently so that a reviewer can trace a custom finding back to the value that produced it.

## 7.3 Configuration and data collection

The configuration loader reads the subscription ID and optional resource-group and storage-account values from the environment or a local configuration file. Live collection requires an authenticated Azure identity and suitable read permissions. Mock mode supplies synthetic records and allows the workflow to be demonstrated without Azure sign-in. Mock data must remain recognisable as mock data in outputs and comparisons. [1, 2]

Inventory queries retrieve resource records, Defender queries retrieve assessment records from SecurityResources, and storage queries retrieve the properties required by the custom rules. The storage collector follows continuation tokens and uses the shared query helper. The resource and Defender collectors require the same structured treatment before complete pagination and reliability can be claimed across all sources. [2]

## 7.4 Finding normalisation and identity

ResourceRecord represents inventory information. AssessmentFinding represents native or custom assessment information, including source, assessment ID, assessed resource ID, status, severity, description, remediation, collection time, raw data and evidence. Additional fields support run identity, first and last observation times, verification status and a separate CloudGuard priority. Keeping priority separate prevents a project ranking from overwriting source severity. [1]

A stable SHA-256-based finding key is derived from the source, original assessment or rule ID, and assessed resource ID. Status and evidence are excluded because those values can change while the finding remains the same. Incomplete identity fields are not suitable for safe deduplication; the persistence design retains the original snapshot payload while excluding such records from the normalised identity layer.

## 7.5 Custom storage rule evaluation

**Table 4 Custom storage rule definitions**

| Rule | Healthy condition | Unhealthy condition | Severity |
|---|---|---|---|
| CG-SA-01 | HTTPS-only is true | HTTPS-only is false | High |
| CG-SA-02 | Public blob access is false | Public blob access is true | High |
| CG-SA-03 | TLS1_2 or TLS1_3 | TLS1_0 or TLS1_1 | Medium |
| CG-SA-04 | ACL default action is Deny | ACL default action is Allow | Medium |

The canonical identifiers are CG-SA-01 through CG-SA-04. For each rule, a missing, null, empty or unrecognised value produces UNKNOWN. The checks normalise recognised boolean strings and case variations, but they do not invent a value when evidence is unavailable. The TLS entries in Table 4 describe what the rule accepts, not a claim that every listed setting is available on the laboratory account. [1, 2]

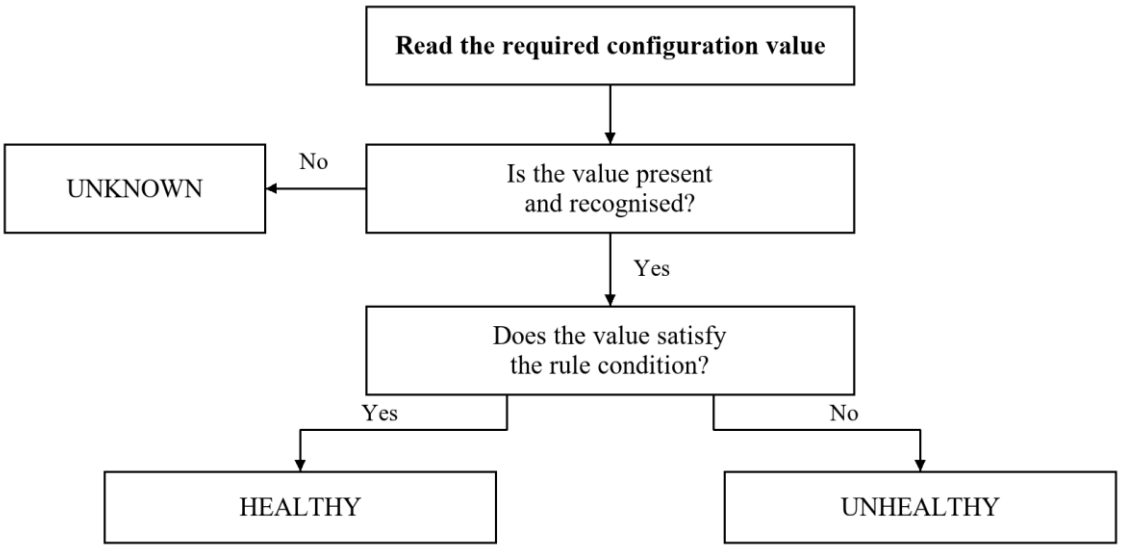



**Figure 2 Evidence-based custom rule decision**

The network ACL rule is a project policy check. An Allow value fails that condition, but it does not prove that anonymous blob reads are enabled or that an attacker accessed data. Those are separate questions with separate evidence.

## 7.6 Collection reliability and error handling

CollectionResult separates retrieval quality from finding status. It records the source, queried scope, UTC start and finish times, retained records, record count, pages fetched, truncation, retry count, sanitised error information and comparison eligibility. Collection states are SUCCESS, PARTIAL, FAILED and SKIPPED. Finding states are HEALTHY, UNHEALTHY, UNKNOWN and NOT_APPLICABLE. A failed query therefore does not become an unhealthy security finding. [1]

The shared Azure helper classifies authentication, authorisation, throttling, timeout and transient-service errors. Selected transient failures use bounded retry, including Retry-After handling and exponential backoff. The documented defaults are three retries and a 30-second request timeout. Permanent permission errors require corrective action rather than repeated requests. These primitives exist, but all collectors and the CLI still need consistent integration. [1, 2]

## 7.7 Snapshot persistence and data relationships

A snapshot stores the run ID, subscription scope, collection time, mode and JSON payloads for the summary, resources, native findings, custom findings and storage properties. The version-2 source design also maintains findings and finding_observations tables. A finding identity can recur across runs, and a run can contain many observations. Figure 3 illustrates the source's version-2 relationships. [2]

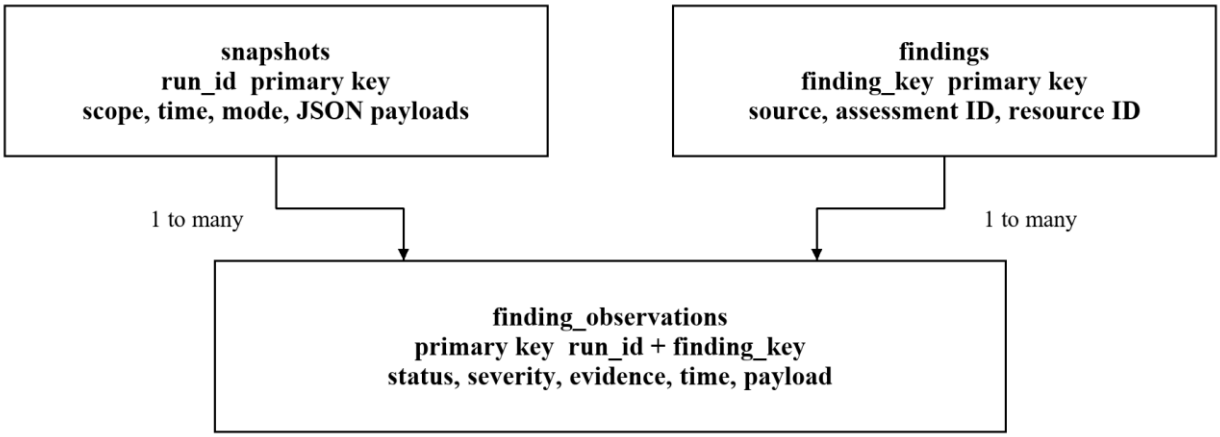



**Figure 3 Version 2 snapshot and finding relationships**

The observation key combines run ID and finding key, connecting a particular finding to a particular scan. The version-1-to-version-2 migration adds the normalised layer and backfills historical JSON records. It does not provide support for the supplied version-3 database. Database selection and schema compatibility must be checked before the stored history is used.

## 7.8 Comparison and export procedure

Comparison matches findings using their source-aware identities and rejects snapshots from different subscription scopes. A finding present only in the current run is new. A previously open finding that receives an eligible healthy observation can be resolved. A previously open finding missing from the later run remains still_open. Existing findings without a verified resolution generally remain unchanged. [1, 2]

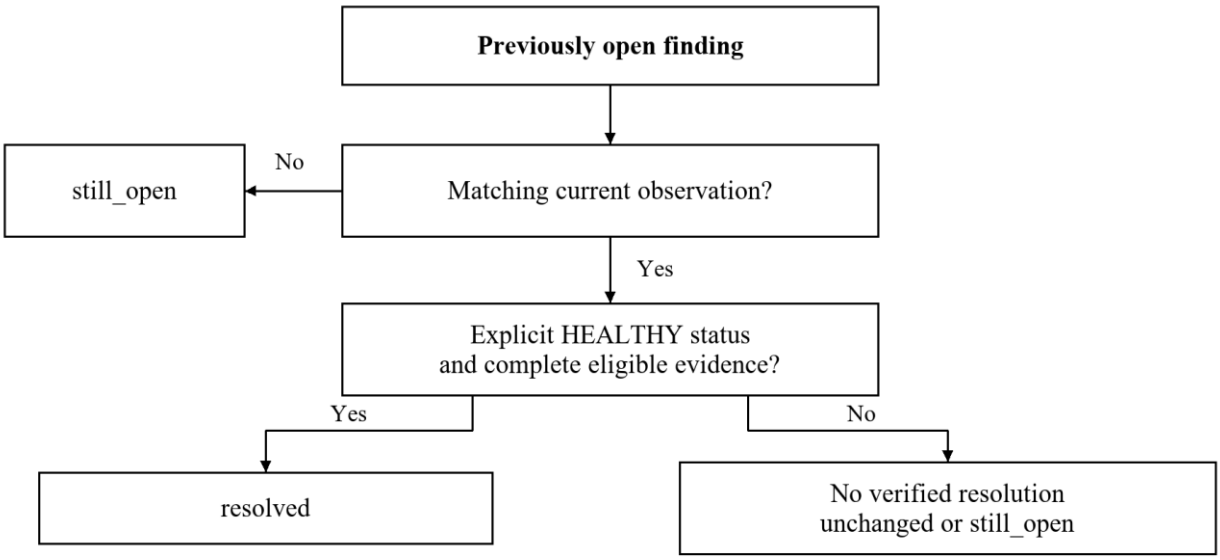



**Figure 4 Comparison decision for an existing open finding**

UNKNOWN and NOT_APPLICABLE are not equivalent to HEALTHY. In the current implementation, a present UNHEALTHY-to-UNKNOWN transition is classified as unchanged, while a missing open finding is still_open. The comparison helper can suppress resolution when completeness flags are false, but the CLI currently uses the defaults rather than source-derived flags. Figure 4 describes the guarded decision the helper supports when it receives correct completeness inputs. [2]

The exporter writes the run IDs, scope, observation times, source, finding identity, classification, previous and current statuses, severity, evidence and remediation information. This allows a reviewer to examine the reason for a classification instead of relying on a single aggregate count.

## 7.9 Verification method

The project uses mocked tests to exercise model construction, collector mapping, rule decisions, persistence and comparison. The historical record also includes consecutive mock runs and saved live Azure evidence. Reviewing these artefacts together helps distinguish a rule's logical correctness from the completeness of a live collection. [1]

The report's evidence review checks the designated backend documentation against current source files, saved JSON and read-only database metadata. It does not constitute a new live Azure scan or a fresh full-suite test run. Current release verification should include a clean dependency installation, two isolated mock runs, schema migration tests and controlled incomplete-collection scenarios before a read-only live validation.

# 8 Novelty

## 8.1 Source-aware assessment records

CloudGuard's principal project-level contribution is the combination of native and custom assessments in a shared record without erasing their origin. A reviewer can see whether Microsoft or a CloudGuard rule produced the result and can follow the actual assessed resource. This avoids a common reporting mistake in a small laboratory: attributing every subscription recommendation to the only storage account in the inventory. [1]

## 8.2 Explicit treatment of uncertainty

The rule engine makes uncertainty visible. For example, a missing minimum TLS value produces UNKNOWN rather than a convenient pass or fail. The collection model addresses a different uncertainty: whether the retrieval itself was complete. Together these concepts provide a basis for more credible historical reporting, provided that their integration is completed.

## 8.3 Traceable observations over time

Stable finding identities and timestamped snapshots make a repeated assessment more informative than a sequence of unrelated exports. Evidence can be traced from the rule result to an observation and then to a run. The conservative comparison design recognises that missing data cannot prove remediation. These are practical engineering contributions within the project; no claim of a novel cryptographic method or a new CSPM research algorithm is made.

# 9 Results/Outcome

## 9.1 Live assessment results

The primary backend documentation records a live run on 30 October 2026 at 07:41:42 UTC. The latest saved run summary in the current live-output directory is dated 30 October 2026 at 18:22:42 UTC and contains the same headline metrics. Table 5 reports those later saved observations. They are historical evidence, not a statement about the subscription's present configuration. [1, 3]

**Table 5 Saved live assessment outcome**

| Measure | Recorded value |
|---|---|
| Resources discovered | 1 |
| Storage accounts inspected | 1 |
| Defender assessments | 22 |
| Unhealthy Defender assessments | 10 |
| Healthy Defender assessments | 4 |
| Not applicable Defender assessments | 8 |
| Custom checks | 4 |
| Healthy custom checks | 3 |
| Unhealthy custom checks | 1 |
| Unknown custom checks | 0 |
| Total findings | 26 |
| Total unhealthy findings | 11 |

The 22 Defender records contain 14 High, five Medium and three Low severity values. Sixteen assessments are subscription-scoped and six are resource-scoped. Severity counts describe the whole Defender set, including records that are healthy or not applicable; they must not be presented as the severity distribution of only the ten unhealthy records. [1, 3]

## 9.2 Custom rule outcome and interpretation

The storage account has HTTPS-only traffic enabled, anonymous public blob access disabled, and minimum TLS set to TLS1_2. These values satisfy CG-SA-01, CG-SA-02 and CG-SA-03. Its network ACL default action is Allow, which fails CG-SA-04. The observed publicNetworkAccess value is Enabled and provides context, but it is not proof of anonymous access. [1, 3]

The immediate review question is whether the account's intended clients require its present network access pattern. A controlled change to a Deny default would need appropriate permitted paths and a follow-up observation. The saved evidence does not show a verified remediation of this finding. It would therefore be inaccurate to describe the project as having secured the account automatically.

## 9.3 Historical testing outcomes

**Table 6 Historical verification milestones**

| Milestone | Reported tests | Coverage and context |
|---|---|---|
| Phase B verification | 119 passed | 100% package coverage reported on 25 October 2026 |
| Phase C hardening | 158 passed | 100% package coverage reported on 26 October 2026 |
| Phase D persistence | 178 passed | Includes comparison and export tests; 26 October 2026 |

The counts in Table 6 are separate milestones and must not be added together. The Phase D handover also describes two consecutive mock runs with 18 findings classified as unchanged in the second run. The consolidated documentation records a live comparison with 26 unchanged findings. Those results involve different input sets and are not interchangeable. [1]

The primary documentation reports that its own fresh test attempt stopped during collection because the Azure SDK was unavailable. No new full-suite pass or current coverage percentage is claimed here. Historical success provides useful context, while release acceptance still requires execution against the exact source and dependency versions being delivered.

## 9.4 Repository outcomes and remaining acceptance criteria

The available artefacts include timestamped JSON evidence, SQLite databases, comparison JSON/CSV files, source modules, tests and a local dashboard. Read-only metadata inspection confirms three snapshots in the schema-3 data database and ten in the schema-2 live-output database. The presence of these artefacts establishes that assessment history has been recorded; it does not prove that the current default execution path can reopen every store. [2, 3]

The backend is therefore best described as an implemented assessment prototype with documented live evidence and specific release defects. The decisive remaining criteria are schema alignment, complete collector integration, safe partial-result handling and comparison driven by real source completeness. A dashboard or an unchanged-finding count cannot substitute for these checks.

# 10 Conclusion

## 10.1 Project contribution

CloudGuard Azure demonstrates how a focused Python application can turn Azure inventory, Defender assessments and custom storage checks into inspectable evidence. The project records what was assessed, where the finding came from and which configuration value supports it. Local snapshots and exports make that evidence available for later review without a separate database service.

The live observations are concrete: one resource, 22 native assessments, four custom checks and one failing custom network ACL condition. The implementation also shows why unknown values and incomplete collection require explicit treatment. A posture assessment becomes misleading if missing evidence is presented as a healthy result or if a finding is declared resolved simply because it no longer appears.

## 10.2 Completion assessment

The project has met much of its functional intent, but the inspected release is not yet internally consistent. The original database exceeds the source's supported schema version, and reliability metadata is not fully carried from collection into comparison. These are bounded engineering tasks with clear verification criteria. Completing them would strengthen the existing workflow more than adding further presentation features at this stage.

The final assessment is therefore a successful demonstration of the core approach, supported by historical tests and saved Azure evidence, with release readiness conditional on the identified corrections. The project remains a read-oriented posture tool; its outputs support a human decision about remediation.

# 11 Recommendation and Future Scope

## 11.1 Immediate release recommendations

First, select the database schema that the release will support and provide a tested migration path. Preserve the existing databases before changing them, and verify that historical snapshots remain readable. The schema-2 live-output store provides a useful compatibility reference, but its presence does not repair the schema-3 default store.

Second, make all three collectors return consistent structured results. Retain records when a later page fails, preserve sanitised diagnostics, and connect source-specific completeness to comparison. A failure in Defender collection must not be masked by a successful inventory query. Tests should verify that incomplete or unknown evidence cannot produce a resolved classification.

Third, align the README with the source that ships, choose a supported Python baseline, and record exact dependency versions after verification. Keep local configuration and generated evidence out of the release unless deliberately sanitised and included. Run the full suite, two mock scans and a controlled live read-only scan against the same release tree. [1, 2]

## 11.2 Controlled remediation validation

A useful next experiment is a before-and-after demonstration of CG-SA-04 in an isolated account. Record the original access requirements, arrange permitted network paths, make the approved configuration change and collect fresh evidence. The comparison should report resolution only after an explicit healthy observation from a complete collection. This would validate the intended lifecycle more convincingly than comparing two unchanged runs.

The experiment should also account for Resource Graph's indexing delay. An immediate follow-up query may still show the previous setting. Record both configuration-change and observation times, and avoid claiming that a stale response proves a failed change. [5]

## 11.3 Broader coverage and usability

Future rules could examine additional storage controls, such as shared-key access and recovery settings, once reliable property collection and policy context are defined. New Azure resource types should be added through the same separation of collector, model and rule logic. Each rule needs explicit healthy, unhealthy and unknown cases, plus evidence and remediation text.

The existing dashboard can develop further by making collection quality and evidence age prominent alongside finding counts. Filtering by source, assessed scope and run would help reviewers avoid comparing unlike records. Accessibility and user testing would provide evidence that the interface is understandable, rather than assuming usability from the presence of charts.

## 11.4 Future evaluation and deployment

A broader evaluation should measure scan duration, query volume, retry behaviour, pagination completeness and database growth across controlled subscription sizes. Rule quality should be evaluated against labelled configuration cases, with false positives and false negatives reported separately. These measurements would establish limits that the one-resource laboratory cannot answer.

Scheduled scans, multi-subscription support and hosted access are possible extensions after the release defects are resolved. Each would introduce additional concerns around identity, permissions, concurrent writes and isolation of run histories. Automated remediation should remain a distinct future capability with approval, rollback and verification requirements, rather than an implied property of the current scan workflow.

# 12 Acknowledgment

## 12.1 Technical articles and community resources

I acknowledge the Microsoft Learn authors whose articles on Cloud Security Posture Management, Azure Resource Graph and Blob storage security helped explain the platform concepts used in this report. Their documentation provides a practical foundation for understanding recommendation scope, configuration visibility and storage security controls. [4, 5, 6]

I also acknowledge the Microsoft Defender for Cloud Blog article, Microsoft Defender for Cloud Data Flow, hosted on Microsoft Community Hub. It provides useful background on the movement and consumption of Defender recommendations beyond the portal. The Python and pytest documentation supported the discussion of local persistence and reproducible testing. The genuine source titles and verified addresses are listed in the references. [8, 9, 10]

## 12.2 Project documentation and open-source tools

I acknowledge the work represented in the CloudGuard backend documentation, phase handovers and saved assessment evidence. These records make it possible to explain both the project's achievements and the issues that remain. I am also grateful to the maintainers of Python, the Azure SDK, SQLite and pytest for the tools on which the implementation depends.

# 13 References

## 13.1 Project materials

[1] CloudGuard Azure. CloudGuard Azure Final Backend Documentation. 30 October 2026. Primary project document: docs/CloudGuard_Azure_Final_Backend_Documentation.md. Historical Phase B, C and D results are attributed to this consolidated record and its cited handovers.

[2] CloudGuard Azure project repository. README.md, requirements.txt, poc_run.py, collector modules, finding and collection models, snapshot_store.py, comparison.py, export.py and frontend files. Source inspection: 1 November 2026.

[3] CloudGuard Azure saved evidence. live-output/evidence/20261030T182242Z_run_summary.json; 20261030T182239Z_defender_assessments.json; 20261030T182242Z_storage_properties.json; 20261030T182242Z_custom_checks.json. Database metadata: data/cloudguard.db and live-output/cloudguard.db. Recorded evidence: 30 October 2026; inspection: 1 November 2026.

## 13.2 Official documentation and technical blog

[4] Microsoft Learn. What is Cloud Security Posture Management CSPM. Accessed 1 November 2026.

https://learn.microsoft.com/en-us/azure/defender-for-cloud/concept-cloud-security-posture-management

[5] Microsoft Learn. What is Azure Resource Graph. Accessed 1 November 2026.

https://learn.microsoft.com/en-us/azure/governance/resource-graph/overview

[6] Microsoft Learn. Security recommendations for Blob storage. Accessed 1 November 2026.

https://learn.microsoft.com/en-us/azure/storage/blobs/security-recommendations

[7] Microsoft Learn. Azure Identity client library for Python. Accessed 1 November 2026.

https://learn.microsoft.com/en-us/python/api/overview/azure/identity-readme?view=azure-python

[8] Python Software Foundation. sqlite3 DB API 2.0 interface for SQLite databases. Accessed 1 November 2026.

https://docs.python.org/3/library/sqlite3.html

[9] pytest documentation. Accessed 1 November 2026.

https://docs.pytest.org/en/stable/

[10] Microsoft Defender for Cloud Blog. Microsoft Defender for Cloud Data Flow. Microsoft Community Hub. Accessed 1 November 2026.

https://techcommunity.microsoft.com/blog/microsoftdefendercloudblog/microsoft-defender-for-cloud-data-flow/1382800